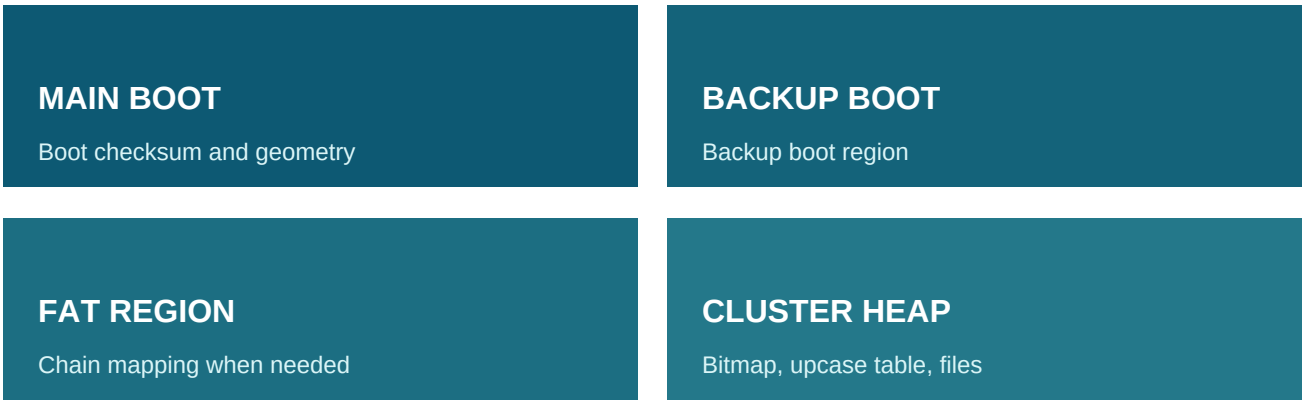
exFAT - Microsoft’s extensible FAT

ORIGINAL FORMAT / HISTORY / ENGINEERING DESIGN

exFAT extends the FAT family for large media with an allocation bitmap, Unicode directory-entry sets and optional contiguous-file operation without FAT chains.

This reference describes the original upstream disk format and its historical engineering rationale. The separate InfiltratorOS adapter is not the original specification. Historical facts, format rules and implementation limitations are separated; consult the named primary sources for field-level normative requirements.

ON-DISK STRUCTURE / CONCEPTUAL MAP



CHAPTERS

1. Historical chronology
2. Original design and on-disk structures
3. Allocation and directories
4. Integrity, validation and recovery
5. Design trade-offs and historical limitations
6. InfiltratorOS interpretation

exFAT - Microsoft’s extensible FAT

HISTORICAL CHRONOLOGY

Microsoft developed exFAT in the 2000s as a successor to FAT32 for larger files and flash-based removable storage. It appeared in Windows Embedded CE 6.0-era systems and spread to consumer removable disks, SDXC cards and desktop operating systems. Microsoft later published the exFAT File System Specification, and the format gained broad cross-platform implementation support.

The central design goals are explicit in Microsoft’s own specification: preserve much of FAT’s implementation simplicity, support much larger files and devices, and permit extensible metadata. exFAT is not FAT32 with a larger file-size field. Its directory entry sets, allocation bitmap and upcase-table semantics require a distinct parser and writer.

exFAT - Microsoft’s extensible FAT

ORIGINAL DESIGN AND ON-DISK STRUCTURES

An exFAT volume has a main boot region, backup boot region, FAT region and data region with a cluster heap. The main boot sector records partition offset, volume length, FAT offset and length, cluster heap offset and count, root directory first cluster, revision, flags and sector/cluster shifts. The boot region includes a checksum; the backup helps retain boot metadata after certain failures.

The cluster heap stores the root directory, allocation bitmap, uppercase conversion table, subdirectories and file content. The first data cluster is numbered two. Cluster index entries are 32 bits, but the bitmap rather than the FAT is authoritative for which clusters are allocated. Directory entry sets combine primary File entries with Stream Extension and File Name secondary entries.

exFAT - Microsoft’s extensible FAT

ALLOCATION AND DIRECTORIES

Allocation bitmap bits indicate used and free clusters. A file may be represented as a contiguous extent without needing a per-cluster FAT chain when the NoFatChain flag applies; fragmented files use the FAT to follow linked clusters. This is a major difference from classic FAT12/16/32 and avoids much table access for contiguous files.

Directories consist of 32-byte records combined into validated entry sets with secondary counts and set checksums. Filenames use UTF-16 code units and a volume upcase table for case-insensitive lookup. The stream extension records the first cluster, data length and valid-data length. Unknown critical entries must not be ignored as though they were benign extensions.

exFAT - Microsoft’s extensible FAT

INTEGRITY, VALIDATION AND RECOVERY

exFAT defines volume-dirty state and checksum rules, but base exFAT does not promise general journalling. TexFAT is a separate transactional extension described in the specification for certain Windows CE uses, with two FATs and two allocation bitmaps. A plain exFAT volume should not be advertised as TexFAT or as providing crash-atomic metadata updates.

Microsoft’s recommended write ordering updates dirty state, the active FAT where necessary, allocation bitmap, and directory entries before clearing the flag. A consistency check must cross-validate the bitmap, any linked FAT chains, directory entry set checksums, file length and contiguous flags. It must preserve unknown benign extension records and reject unknown critical ones.

exFAT - Microsoft’s extensible FAT

DESIGN TRADE-OFFS AND HISTORICAL LIMITATIONS

exFAT supports 64-bit file lengths and much larger clusters than the original FAT variants, and its allocation bitmap decouples free-space tracking from chain traversal. Larger permissible clusters improve scalability but also increase possible internal fragmentation; the chosen geometry must respect device and interoperability limits.

The format avoids the complexity of a tree-indexed or pooled copy-on-write filesystem, favouring portable storage. It does not inherently provide Unix ACL semantics, filesystem-wide snapshots or cryptographic end-to-end checksums. Filesystem consistency under abrupt removal remains a responsibility of implementation write ordering and repair utilities.

exFAT - Microsoft’s extensible FAT

INFILTRATOROS INTERPRETATION

The InfiltratorOS exFAT.so module interprets the Microsoft exFAT disk format, independently of the FAT12/16/32 modules. Historical specifications define its on-media requirements; our build and test system defines only what the current implementation demonstrably supports.

A valid implementation must not mistake a contiguous exFAT stream for a FAT32 cluster chain. It must honour the upcase table, bitmap and entry set checksums, and distinguish ordinary exFAT from TexFAT. Installation status of the shared object or kernel adapter is not proof of recovery qualification.

exFAT - Microsoft’s extensible FAT

PRIMARY SOURCES & DOCUMENTATION

References are included to make the historical and technical account auditable. The source documents, published format specifications and implementation references take precedence over this explanatory guide where a detail differs.

1. Microsoft exFAT File System Specification (authoritative normative design)

https://learn.microsoft.com/en-us/windows/win32/fileio/exfat-specification

2. Microsoft FAT Specification (predecessor family comparison)

https://www.scs.stanford.edu/~zyedidia/docs/_other/fat.pdf

Implementation boundary: the installed InfiltratorOS .so, the optional Linux kernel .ko, and the source-first installer are distinct products. The original filesystem design is documented above; claims about the adapter’s qualification require its own source, tests and release evidence.

exFAT / technical detail

ON-DISK FIELD AND STRUCTURE INVENTORY

Format-specific field names and meanings are summarised below; where applicable the original specification defines exact endianness, offsets, checksums and format-revision limits.

1. Main Boot Sector: geometry, revision, flags, cluster heap.
2. Backup Boot Region: secondary copy and checksum structures.
3. FatOffset/FatLength: placement of optional cluster-chain links.
4. ClusterHeapOffset/ClusterCount: allocation region bounds.
5. Allocation Bitmap: authoritative cluster free/used information.
6. Up-case Table: mapping for volume name comparison.
7. File Entry set: primary, Stream Extension and File Name entries.
8. NoFatChain: contiguous stream allocation without FAT traversal.

exFAT / technical detail

HISTORICAL FORMAT DEVELOPMENT

Selected design milestones explain why this format acquired its present layout. Dates refer to historical format or implementation changes, not InfiltratorOS releases.

1. 2000s: Microsoft develops exFAT for large portable storage.

2. 2006 era: exFAT arrives in Windows Embedded CE.

3. 2010s: SDXC media and desktop software adopt exFAT broadly.

4. 2019: Microsoft publicly publishes detailed exFAT specification.

Important historical distinction: newly introduced driver features are not necessarily changes to the original on-disk format. Read the version or feature bits rather than assuming current software support defines historic media.

FORMAT VALIDATION AND RECOVERY INVARIANTS

A writable implementation must establish all of these conditions before modifying any original-format media. These checks describe the minimum class of invariants, not proof of complete recovery qualification.

1. Verify main and backup boot checksums separately.

2. Do not treat FAT as authoritative for free clusters.

3. Validate entry set checksum and required secondary records.

4. Respect critical versus benign unknown directory entry semantics.

5. Use dirty flag and specified metadata write ordering.

Forensic principle: use a separate verified image, retain original structures, and report conflicting evidence instead of inventing a new supposedly clean filesystem state.

exFAT / engineering detail

DISK READ PATH / ENGINEERING NOTES

1. Read both the Main and Backup Boot Regions, including volume length, FAT offset and length, Cluster Heap offset, cluster count and root-directory first cluster.

2. Validate the exFAT boot checksum and the boot-region rules before accepting geometry. The boot region contains different structures from a FAT32 BIOS Parameter Block.

3. Use the Allocation Bitmap as the primary free-space map. A FAT chain is not required for every allocated file extent in exFAT.

4. Locate the Up-case Table through its root-directory entry, validate the table checksum and use its mapping for format-defined case-insensitive names.

5. Read file directory-entry sets consisting of a primary File entry, Stream Extension and one or more File Name entries.

6. Validate entry-set checksums and secondary-entry counts before using stored file lengths, name hashes or first-cluster references.

7. Interpret the NoFatChain flag for contiguous file allocation; a file may not use ordinary FAT next-cluster traversal even though a FAT exists.

8. Translate cluster number two to the first cluster in the heap. Honour sectors-per-cluster and the volume boundaries from the boot record.

exFAT / engineering detail

VALIDATION, RECOVERY AND COMPATIBILITY

1. Validate both boot regions independently; disagreement calls for evidence-based recovery rather than blindly preferring the first copy.

2. Check entry-set checksums, name lengths and the number of secondary entries. An incomplete or out-of-order entry set must not create a fictitious file.

3. Compare Allocation Bitmap state against referenced file extents. The FAT is not an authoritative allocation bitmap in exFAT.

4. For NoFatChain files, validate that the contiguous cluster range fits both declared file length and heap bounds.

5. Distinguish DataLength and ValidDataLength. Reads beyond valid initialized content must not reveal stale disk data.

6. Do not reuse FAT32 FSInfo, short-name aliases or fixed-root-directory assumptions; exFAT defines different boot and directory structures.

7. Use the VolumeDirty and media-failure flags as signals requiring further validation, not as substitutes for checking actual directory and allocation state.

8. Preserve directory metadata, the Up-case Table and both boot regions when attempting repairs; changing a case map can alter the meaning of existing filenames.